Native Speed, Java Comfort

Calling Rust from the JVM with Project Panama

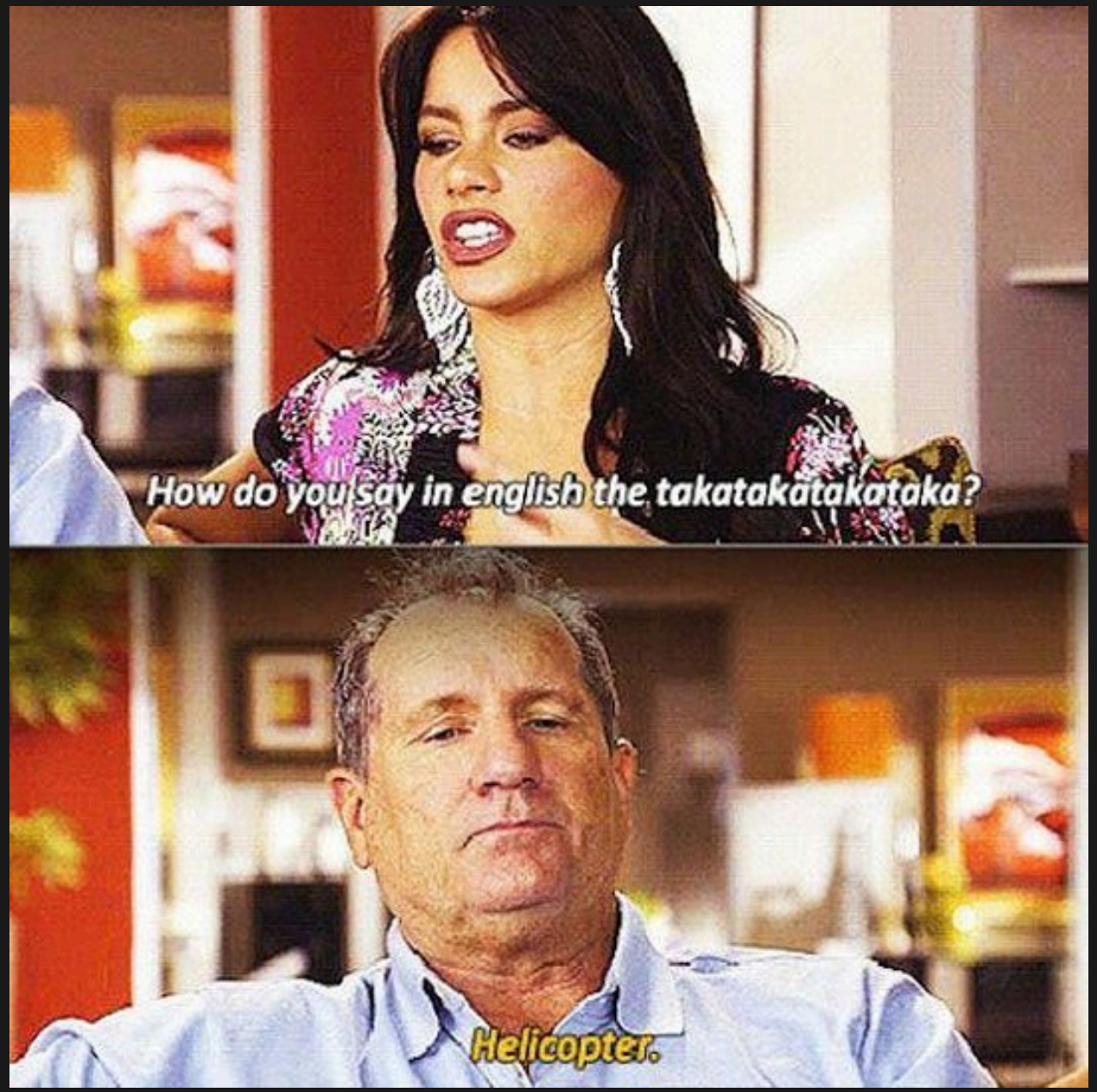
Gonzalo Ortiz · Devoxx UK

Apache Pinot / StarTree

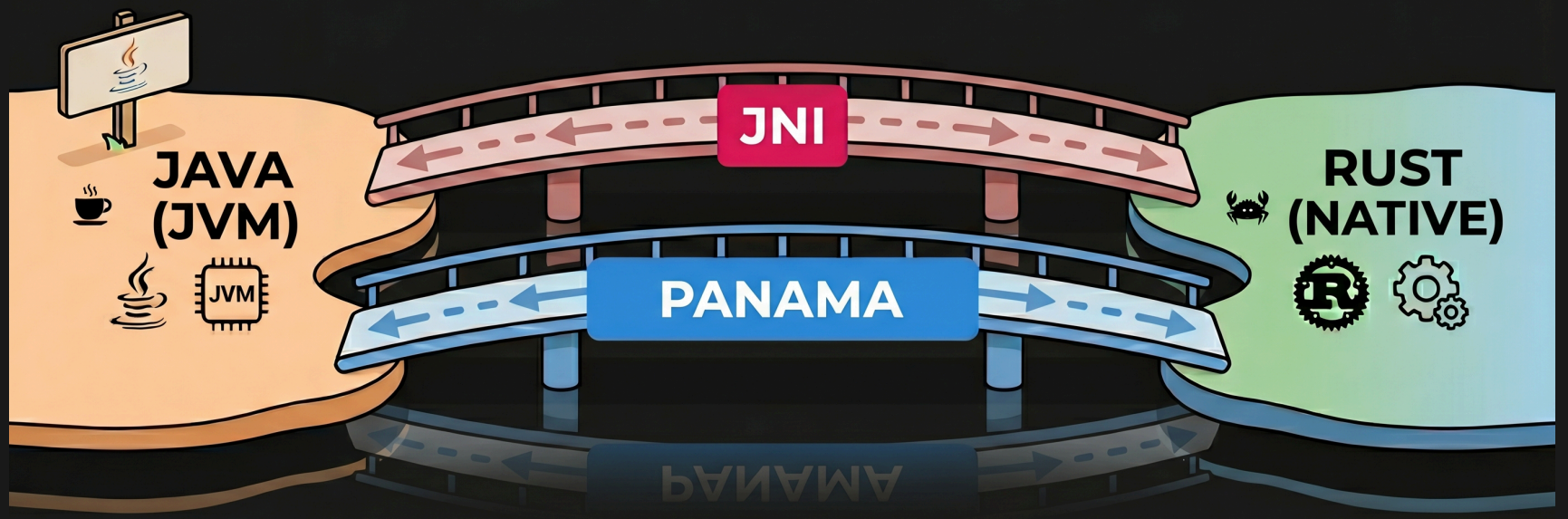
A tale of two barriers

- The isthmus that separates Java from the native world
- My Spanish accent


Do you even know how smart I am in Spanish?


How do you say in english the takatakatakataka?
Helicopter.

The two bridges



JNI — in the JDK since Java 1.1 · **Panama FFM** — finalized in Java 22

Same call, two philosophies

JNI — object-centric

```toml
[lib]
crate-type = ["cdylib"]

[dependencies]
jni = "0.21"
```

- Rust receives Java objects
- JVM may pin objects
- Rust crate coupled to the JVM

Panama — mem-centric

```toml
[lib]
crate-type = ["cdylib"]
```

- Rust receives raw memory
- No GC interaction — data lives off-heap
- Same standard C ABI
  - Already used by Python, Go, Node…

Why call native code?

- **Ecosystem access** — reuse Rust/C/C++ libraries that already exist.
  ML inference (ONNX), compression (zstd), databases. Don't rewrite them.
- **Ergonomics** — some problems are more natural with direct memory control.
  SIMD intrinsics, memory-mapped structures, zero-copy I/O.
- **Performance** — bypass JIT/GC limits…
  Be careful — the JVM is surprisingly good. Native isn't a magic speed button.

add(a, b)

JNI



```rust
#[no_mangle]
pub extern "system"
fn Java_com_example_Lib_add(
    _env: JNIEnv, // Used to access the JVM
    _class: JClass, // Our "Lib" class
    a: jint,
    b: jint,
) -> jint {
    a + b
}
```



```java
public class Lib {
    static { System.loadLibrary("mylib"); }
    public static native int add(int a, int b);
}

int r = Lib.add(2, 3);   // → 5
```

Panama

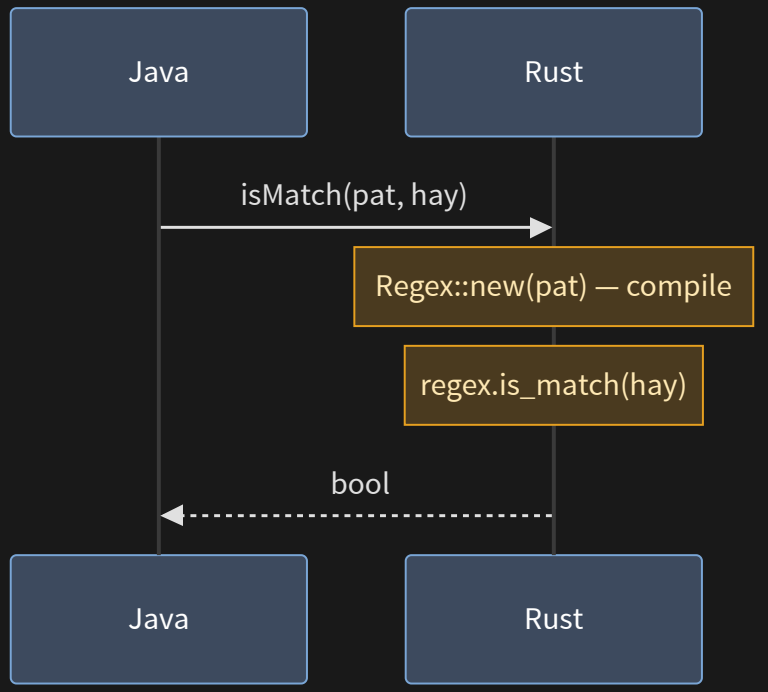


```rust
#[no_mangle]
pub extern "C"
fn add(a: i32, b: i32) -> i32 {
    a + b
}
```



```java
private static final MethodHandle ADD;
static {
    SymbolLookup lib = SymbolLookup.libraryLookup(
        "libmylib.so", Arena.global());
    ADD = Linker.nativeLinker().downcallHandle(
        lib.find("add").orElseThrow(),
        FunctionDescriptor.of(
            JAVA_INT, JAVA_INT, JAVA_INT));
}
int r = (int) ADD.invoke(2, 3);   // → 5
```

Using strings



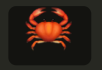
isMatch(pat, hay) — Rust

JNI



```rust
#[no_mangle]
pub extern "system"
fn Java_org_example_rust_JniRsRegex_isMatch(
    mut env: JNIEnv,
    _class: JClass,
    pattern: JString, // pointer to Java String
    haystack: JString, // pointer to Java String
) -> jboolean {
    let pat: String = env.get_string(&pattern)
      .unwrap().into();
    let hay: String = env.get_string(&haystack)
      .unwrap().into();
    Regex::new(&pat).unwrap().is_match(&hay)
      as jboolean
}
```

Panama



```rust
#[no_mangle]
pub extern "C" fn is_match_once(
    pat_ptr: *const u8, pat_len: usize,
    hay_ptr: *const u8, hay_len: usize,
) -> bool {
    let pat = unsafe {
      std::str::from_utf8_unchecked(
        std::slice::from_raw_parts(
          pat_ptr, pat_len)) };
    let hay = unsafe {
      std::str::from_utf8_unchecked(
        std::slice::from_raw_parts(
          hay_ptr, hay_len)) };
    Regex::new(pat).unwrap().is_match(hay)
}
```

isMatch(pat, hay) — Java

JNI

```java
public class JniRsRegex {
    static {
        System.loadLibrary("jrs_regex");
    }

    private static native boolean isMatch(
        String pat, String hay);
}

// Usage
boolean m = JniRsRegex.isMatch(
    "[a-z]+@[a-z]+\.com", "user@example.com");
```

Panama

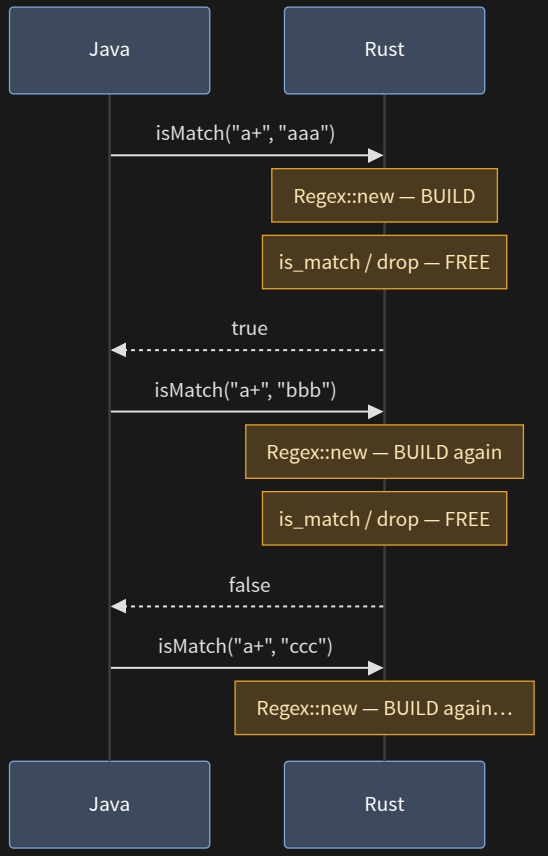
```java
static final MethodHandle IS_MATCH = Linker
          .libraryLookup(...)
          .downcallHandle(...);

public boolean isMatch(String pat, String hay) {
    try (Arena arena = Arena.ofConfined()) {
        MemorySegment cP = arena.allocateFrom(
            JAVA_BYTE,
            pat.getBytes(UTF_8);
        MemorySegment cH = arena.allocateFrom(
            JAVA_BYTE,
            hay.getBytes(UTF_8));
        return (boolean) IS_MATCH.invoke(
            cP, pat.length(), cH, hay.length());
    }
}
```

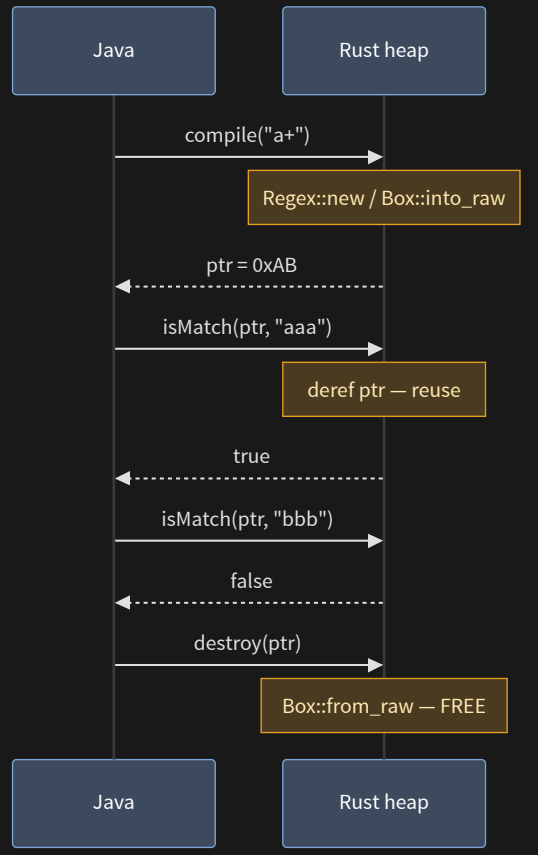
⚠ Hidden bug — spot it before the next slide

Compile every call → compile once

✗ Problem: rebuild on every call



✓ Solution: compile once, pass a handle



Compiled Regex

```rust
pub struct RegexHandle { regex: Regex }

#[no_mangle]
pub extern "C" fn create(
    pattern: *const u8, len: usize,
) -> *mut RegexHandle {
    let pat = bytes_to_str(pattern, len);
    Box::into_raw(Box::new(RegexHandle {
        regex: Regex::new(pat).unwrap(),
    }))
}

#[no_mangle]
pub extern "C" fn is_match(
    handle: *mut RegexHandle,
    hay: *const u8,
    len: usize,
) -> bool {
    if handle.is_null() { return false; }
    unsafe {
      let h = &*handle;
      h.regex.is_match(bytes_to_str(hay, len))
    }
```

```java
public class PanamaRsRegex implements AutoCloseable {
    private final MemorySegment handle;
    private final Arena arena;

    public PanamaRsRegex(String pattern) {
        arena = Arena.ofConfined();
        byte[] utf8 = pattern.getBytes(UTF_8);
        MemorySegment cPat =
            arena.allocateFrom(JAVA_BYTE, utf8);
        handle = (MemorySegment)
            CREATE.invoke(cPat, cPat.byteSize());
    }

    public boolean isMatch(String haystack) {
        try (Arena local = Arean.ofConfined()) {
            MemorySegment cHay =
                arena.allocateFrom(...);
            return (boolean) IS_MATCH.invoke(
                handle, cHay, cHay.byteSize();
        }
    }

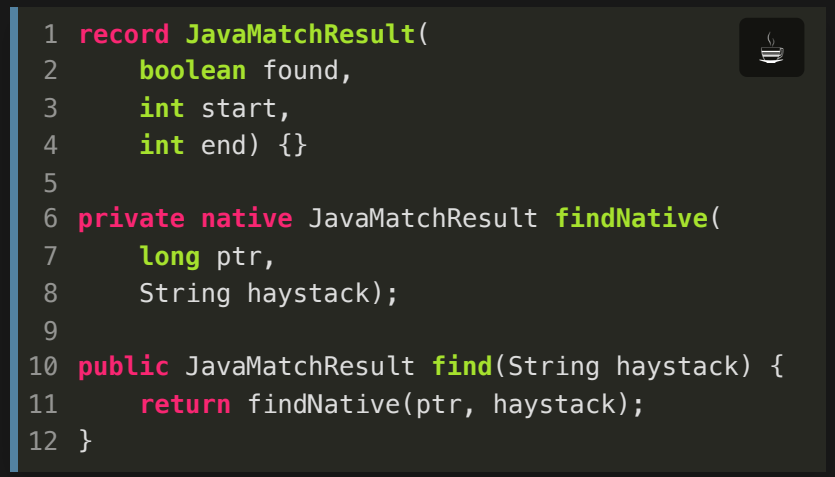
    @Override
```

⚠ Hidden bug — spot it before the next slide

Problem: returning multiple values

- `find()` should return: did it match? start offset? end offset?
- C ABI: one return value → needs a struct
- JNI — Rust can construct Java objects
- Panama — Rust has no JVM access
  - → **return a Rust struct**

Returning a struct: JNI



```java
record JavaMatchResult(
    boolean found,
    int start,
    int end) {}

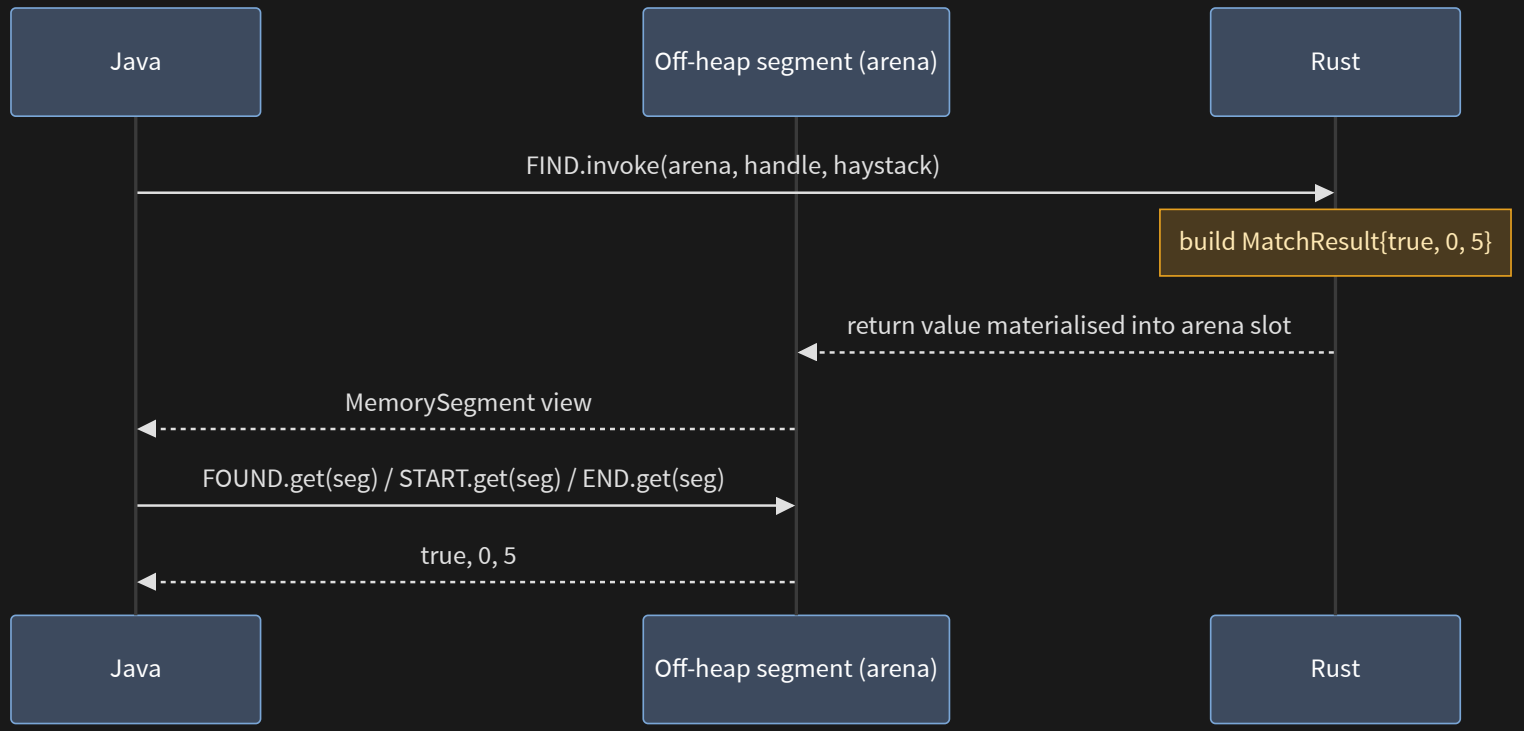
private native JavaMatchResult findNative(
    long ptr,
    String haystack);

public JavaMatchResult find(String haystack) {
    return findNative(ptr, haystack);
}
```



```rust
#[no_mangle]
pub extern "system" fn Java_..._findNative(
    mut env: JNIEnv, _class: JClass,
    regex_ptr: jlong, haystack: JString,
) -> jobject {
    let regex: Regex = ...;
    let hay: String = ...;
    let cls  = env
        .find_class("org/example/JavaMatchResult")
        .unwrap();
    let ctor = env
        .get_method_id(cls, "<init>", "(ZII)V")
        .unwrap();
    match regex.find(&hay) {
        None => env.new_object(cls, ctor,
            &[false.into(), 0.into(), 0.into()]
        ).unwrap().into_raw(),
        Some(m) => env.new_object(cls, ctor,
            &[true.into(), m.start().into(), m.end().int
        ).unwrap().into_raw(),
    }
}</init>
```

In Panama?


Java
Off-heap segment (arena)
Rust
FIND.invoke(arena, handle, haystack)
build MatchResult{true, 0, 5}
return value materialised into arena slot
MemorySegment view
FOUND.get(seg) / START.get(seg) / END.get(seg)
true, 0, 5
Java
Off-heap segment (arena)
Rust

Returning a struct: Panama

```rust
#[repr(C)]
pub struct MatchResult {
    pub found: bool,
    pub start: i32,
    pub end:   i32,
}

#[no_mangle]
pub extern "C" fn find(
    handle: *mut RegexHandle,
    hay_ptr: *const u8, hay_len: usize,
) -> MatchResult {
    let hay = bytes_to_str(hay_ptr, hay_len);
    match unsafe { &*handle }.regex.find(hay) {
        Some(m) => MatchResult {
            found: true,
            start: m.start() as i32,
            end:   m.end()   as i32,
        },
        None => MatchResult { found: false, start:
    }
}
```

```java
static final StructLayout MATCH_RESULT = MemoryLayo
  JAVA_BOOLEAN.withName("found"),
  JAVA_INT.withName("start"),
  JAVA_INT.withName("end"));
static final VarHandle FOUND =
  MATCH_RESULT.varHandle(groupElement("found"));
static final VarHandle START =
  MATCH_RESULT.varHandle(groupElement("start"));
static final VarHandle END   =
  MATCH_RESULT.varHandle(groupElement("end"));

static final MethodHandle FIND =
  Linker.nativeLinker().downcallHandle(...)

public JavaMatchResult find(String haystack) {
  try (Arena arena = Arena.ofConfined()) {
    byte[] utf8 = ...;
    MemorySegment cHay = ...;
    MemorySegment out = (MemorySegment) FIND
      .invoke(
        arena, handle, cHay, cHay.byteSize());
    if (!(boolean) FOUND.get(out, 0L))
```

⚠ Hidden bug — spot it before the next slide

The padding trap

❌ What we just wrote

```
structLayout(
  JAVA_BOOLEAN, // offset 0, 1B
  JAVA_INT,     // offset 1 ← WRONG!
  JAVA_INT      // offset 5 ← WRONG!
)
```

✓ Correct

```
structLayout(
  JAVA_BOOLEAN,    // offset 0, 1B
  paddingLayout(3),// 3B padding
  JAVA_INT,        // offset 4 ✓
  JAVA_INT         // offset 8 ✓
)
```

From Rust to Java bindings

Rust source → cbindgen → regex_h.h → jextract → Java bindings

1. Generate a C header from Rust

```
cargo install cbindgen
cbindgen --crate jrs-regex-panama --output regex_h.h
```

2. The generated header

```h
// regex_h.h — generated by cbindgen
struct MatchResult { bool found; int32_t start; int32_t end; };

struct RegexHandle *create(
    const uint8_t *pat_ptr, uintptr_t pat_len);
bool is_match(const struct RegexHandle *handle, ...);
void destroy(struct RegexHandle *handle);
```

From Rust to Java bindings

Rust source → cbindgen → regex_h.h → jextract → Java bindings

3. Auto-generate Java bindings

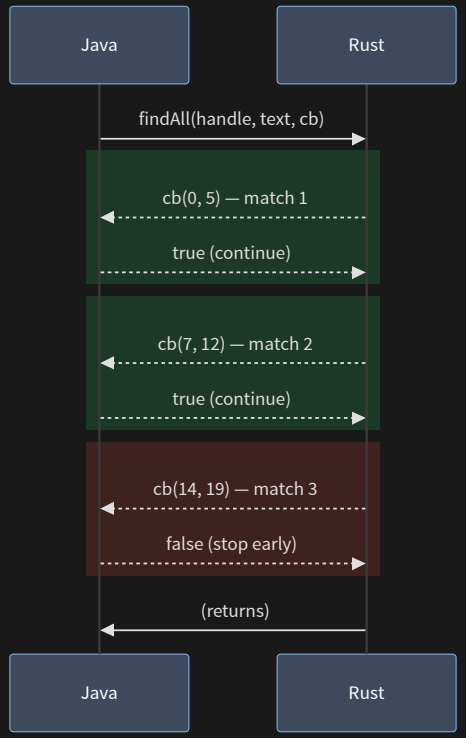
```
jextract --output src/main/java \
  --target-package org.example.rust.generated \
  regex_h.h
```



```java
// Generated by jextract — you don't write this by hand
public static MethodHandle is_match =
    LINKER.downcallHandle(
        LOOKUP.find("is_match").orElseThrow(),
        FunctionDescriptor.of(JAVA_BOOLEAN, ADDRESS, ADDRESS, JAVA_LONG));

public static StructLayout MatchResult = MemoryLayout.structLayout(
    JAVA_BOOLEAN.withName("found"),
    MemoryLayout.paddingLayout(3),   // jextract gets the padding right
    JAVA_INT.withName("start"),
    JAVA_INT.withName("end"));
```

Callback/upcalls



Callbacks: the Java contract



```java
// The interface Rust will call back into
interface MatchVisitor {
    // return true to continue, false to stop early
    boolean onMatch(int start, int end);
}

// Usage
regex.findAll(haystack, (start, end) -> {
    System.out.println("match at " + start + ".." + end);
    return true;  // keep going
});
```

Callbacks: JNI



```rust
#[no_mangle]
pub extern "system" fn Java_..._forEachMatch(
    mut env: JNIEnv, _class: JClass,
    ptr: jlong, haystack: JString,
    visitor: JObject,   // the MatchVisitor instance
) {
    let regex = unsafe { &*(ptr as *const Regex) };
    let haystack_str: String = env.get_string(&haystack).unwrap().into();
    for m in regex.find_iter(&haystack_str[..]) {
        let keep_going = env.call_method(
            &visitor,          // Java object
            "onMatch",         // method name (string!)
            "(II)Z",           // JNI type signature (string!)
            &[JValue::from(m.start() as jint),
              JValue::from(m.end() as jint)]
        ).expect("onMatch call failed")
         .z().expect("must return boolean");
        if !keep_going { break; }
    }
}
```

Callbacks: Panama



```rust
// Plain C function pointer — no Java types
pub type MatchCallback =
  unsafe extern "C" fn(
    start: i32, end: i32
  ) -> bool;

#[no_mangle]
pub extern "C" fn for_each_match(
    handle: *mut RegexHandle,
    hay_ptr: *const u8, hay_len: usize,
    callback: MatchCallback, // <- the function pointer
) {
    let hay = bytes_to_str(hay_ptr, hay_len);
    for m in unsafe { &*handle }.regex.find_iter(hay) {
        // indexes are not correct (UTF-8 vs UTF-16)
        if !callback(m.start() as i32, m.end() as i32) {
            break;
        }
    }
}
```

A plain C function pointer. No mention of Java. This library could be called from Python, Go, or Java — it doesn't care.

Callbacks: Panama

```java
// The Java method that will be called back
boolean onMatch(int start, int end) {
    System.out.println("match at " + start + ".." + end);
    return true;
}

// Wire it into a native function pointer — done once at setup
static final MethodHandle MATCH_UPCALL_BRIDGE =
    MethodHandles.lookup().findVirtual(
        PanamaRsRegex.class,
        "onMatch",
        MethodType.methodType(boolean.class, int.class, int.class));

// The downcall handle for for_each_match
static final MethodHandle FOR_EACH_MATCH = LINKER.downcallHandle(
    LIB.find("for_each_match").orElseThrow(),
    FunctionDescriptor.ofVoid(
        ADDRESS,              // handle
        ADDRESS, JAVA_LONG,   // haystack ptr + len
        ADDRESS               // callback function pointer
```

Callbacks: Panama



```java
public void forEachMatch(String haystack) {
    try (Arena arena = Arena.ofConfined()) {
        MemorySegment stub = LINKER.upcallStub(
            MATCH_UPCALL_BRIDGE,
            FunctionDescriptor.of(
                JAVA_BOOLEAN, JAVA_INT, JAVA_INT),
            arena);
        byte[] utf8 = haystack.getBytes(UTF_8);
        MemorySegment cHay = arena.allocateFrom(JAVA_BYTE, utf8);
        FOR_EACH_MATCH.invoke(handle, cHay, (long) utf8.length, stub);
    }
}
```



Wrapping Up

What we built today

Semantic case	JNI	Panama
Simple call with primitives	✓	✓
Simple call with strings	✓	✓
Return a native handle (pattern ref)	✓	✓
Return an object / struct	✓	✓
Callback / upcall from native	✓	✓

When should you reach for native?

Reason	Worth it?	Notes
Ecosystem	✓ Yes	Now it is easier to use existing libraries
Ergonomics	✓ Yes	Genuine JVM blind spots
Performance	⚠ Verify first	JIT/GC are good. Measure

Performance tips

- Each call is expensive.
  - JNI slower than Panama
- One broad > many narrow.
- Shared memory > copies.
- Do not use Java Strings.
- Use async-profiler — not JFR.

Toolbox

- **Build** — `rust-maven-plugin` bundles native libs inside the jar.
- **Debugging** — Two debuggers, two windows.
- **Logging** — Rust `log` / `tracing` don't feed SLF4J.
- **IDEs & AI** — No cross-boundary refactoring. AI shines for translation.
- **Two heaps, one process** — GC knows nothing about Rust allocations.



One last critical part

⚠ A panic will kill the JVM

```
let regex = Regex::new(&pattern).expect("Invalid regex");
//                               ^^^^^^^ panic on invalid input!
```



A panic across `extern "C"` is **undefined behavior**.
No stack trace. No exception. The JVM just dies.

Catch panics at the boundary



```rust
// Rust: catch the panic, signal failure with a null pointer
pub extern "C" fn create(
    pattern_ptr: *const u8, pattern_len: usize,
) -> *mut RegexHandle {
    let result = std::panic::catch_unwind(|| {
        let pattern = bytes_to_str(pattern_ptr, pattern_len)?;
        Ok(Regex::new(pattern)?)
    });
    match result {
        Ok(Ok(regex)) => Box::into_raw(Box::new(RegexHandle { regex })),
        _ => std::ptr::null_mut(), // in case of error
    }
}
```

Alternative to returning `null`? Why not *throw an exception*?

Resources

- `github.com/gortiz/jrs-regex` — full project, benchmarks, flamegraphs
- JEP 454 — Foreign Function & Memory API
- `docs.rs/regex` — Rust regex crate
- `github.com/mozilla/cbindgen`
- `github.com/openjdk/jextract`
- async-profiler, JMH

Thank you

Questions?

Gonzalo Ortiz · Devoxx UK